

REPORT_21

This report will talk about a few different communication protocols

I2C PROTOCOL:

This communication protocol works on only two buses. A serial clock line and a serial data line.

Data on the serial data line is sent in the following format:

Start condition, 7 to 1 or 10 to 1 bits for address of slave, read or write bit(1 or 0 respectively), ack/nack bit, register address bits, ack/nack bit, data bits, ack/nack bit, end condition. (according to www.nxp.com, there is no register address in the data frame, however according to other sites, register address is also sent).

The i2c is an active low protocol, that is, when the bus is at 0V, it is considered as logic 1 and hence has to be pulled up to a High voltage continuously. Acknowledge signal is clock pulse. If when the acknowledge pulse is sent and the SDA is kept high by the slave, then it is considered to not be acknowledged, else it is considered acknowledged by the slave.

The start and stop signals are also generated in a similar way, when the clock signal is held high and the SDA signal line is made to fall, a start condition is obtained. But when the SDA signal is high, the end condition is achieved.

On an arduino, I2c bus is used in many places(often using the wire.h library) and the clock and data pins on the arduino (analog 4 and 5).

SPI PROTOCOL:

SPI stands for Serial peripheral interface protocol. This is a form of communication which uses serial data for data transfer. Usually has 4 pins, Master output, slave input, master input, slave output, slave select(can be multiple pins to deal differently with different slaves) and the clock pin. For data transfer, the master first takes the slave select line for the particular slave and makes it low (it is high) and then sends the clock pulse. The MOSI(master output slave input pin) is used to write to the slave and the MISO(master input, slave output) is used to read from the slave.

The basic steps are:

Sending of clock pulse

Making the SS/CS line low to activate the slave/chip connected to it

Sending of data one bit at a time

Receiving data one bit at a time(if needed).

Unlike in I2C, there are no interruptions during data transfer which is continuous and not packaged (no need for addresses as well).

SPI protocol has this one advantage over I2C and UART.

UART PROTOCOL:

UART stands for universal asynchronous receiver transmitter. That means it requires no clock line and it uses only two wires with only one master and one slave.

One thing to know in UART is that the two data lines are called Rx and Tx. the Rx of the master is connected to the Tx of the slave and vice versa

UART basically converts data from a parallel form to serial data.

When data is to be sent through the UART, it is in parallel form. The data is made a packet along with a start bit, a parity bit(for error checking) and an end bit. The UART data line is active high and hence a 0 signifies start of data transmission. When the UART at the slave detects this change, it gets ready to receive the data. The data frame consists of the data and the parity bit thus sending from 5 to 8 bits of data at once(LSB is sent first).

The UART on both sides also have to have the same baud rate or transmission speed with an acceptable error of 10%

One major advantage is that there is no clock signal unlike in I2C and SPI, however, there can only be one master and one slave for this protocol.

Like I2C, Rx and Tx on the arduino have special pins which are digital 0 and 1 respectively.

USART:

The universal synchronous asynchronous receiver transmitter is very similar to UART but can also be made to run in the synchronous mode wherein it generate a clock cycle with which the data is transferred in blocks/packages and the receiver is synchronised using the clock generated by the transmitter. This greatly increases the data transfer rate and also opens up different ways of transferring data through the same IC.

CODES:

ULTRASONIC WITH PULSEIN():

```
int trigPin = 8;
int echoPin = 9;
long t;
int distance;
void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  Serial.begin(9600);
}
```

```

void loop() {
  digitalWrite(trigPin, LOW);
  digitalWrite(trigPin, HIGH);
  //delayMicroseconds(10); //works with and without delay
  digitalWrite(trigPin, LOW);
  t = pulseIn(echoPin, HIGH);
  distance= (t*0.034)/2;
  if(distance>100){Serial.println("out of range");}
  else
  {
    Serial.print("Distance: ");
    Serial.println(distance);
  }
  delay(200);
}

```

ULTRASONIC WITHOUT PULSEIN():

```

int trig = 9;
int echo = 10;
long val;
long t1,t2;
long t;
void setup()
{
  pinMode(trig, OUTPUT);
  pinMode(echo, INPUT);
  Serial.begin(9600);
}
void pulse()
{
  digitalWrite(trig, LOW);
  digitalWrite(trig, HIGH);
  //delayMicroseconds(10);
  digitalWrite(trig, LOW);
  while(digitalRead(echo)==LOW) //takes around 470 microseconds to execute,
with approx 66 iterations
  {}
  Serial.println(t);
  Serial.println(digitalRead(echo));
}

```

```

    t1 = micros();

    while(digitalRead(echo) == HIGH)
    {
        Serial.println(digitalRead(echo));
        t2 = micros();
        val = (t2-t1)*0.034/2;
        if(val>100){Serial.println(" out of range " );}
        else
        {
            Serial.print(val);
            Serial.println("cm");
            delay(500);
        }
    }
    void loop()
    {
        pulse();

```

```

    }

```

ULTRASONIC WITH LED(DISTANCE MAPPED TO BINARY NUM):

```

int LED[50];
int switchpin = 4;
int i = 0;
int count = 0;
int lev;
bool current = 0; bool last = 0;
int trig = 9;
int echo = 10;
long t1,t2;
long val;
int temp;

```

//this was the code in which i had the hardware issue of LED turning on only when i touch it, also if the obstacle is too close, the LEDs glow to represent 7

//The glow is also dim

```

void pulse()
{

```

```

    digitalWrite(trig, LOW);

```

```

    digitalWrite(trig, HIGH);
    //delayMicroseconds(10);
    digitalWrite(trig, LOW);

    while(digitalRead(echo)==LOW)
    {}

    t1 = micros();

    while(digitalRead(echo) == HIGH)
    {}

    t2 = micros();
    val = (t2-t1)*0.034/2;
    if(val>100){Serial.println(" out of range " );delay(500);}
    else
    {
        Serial.print(val);
        Serial.println("cm");
        delay(500);
    }
}

void dmap(long dist)
{
    dist = constrain(dist, 0, 100);
    lev = map(dist, 0, 100, 0, 5);
    Serial.print(" num: ");
    Serial.println(lev);
}

void debounce()
{
    current = digitalRead(switchpin);
    if( current != last)
    {
        delay(5);current = digitalRead(switchpin);

    }
}

void LEDcount(int lev)

```

```

{

{
    for(int s=0;s<3;s++)
    {

        digitalWrite( LED[s], 0);

    }
    int a[50];
    temp = lev;
    Serial.print("temp: ");Serial.println(temp);
    int j = 0;
    while(temp != 0)
    {
        a[j]=temp%2;

        temp = temp/2;
        j++;
    }
    for(int r=0;r<j;r++)
    {

        digitalWrite( LED[r], a[r]);

        Serial.print("LED[");Serial.print(r);Serial.print("]: ");
        Serial.println(digitalRead(LED[r]));
    }
    Serial.print("j: ");Serial.println(j);

    if(temp>5){ temp = 0;}
}

delay(500);
}
void setup()
{
    //int LED[50];
    pinMode(trig, OUTPUT);

```

```

pinMode(echo, INPUT);
pinMode(03, OUTPUT);
Serial.begin(9600);
LED[0]=5;LED[1]= 6;LED[2] = 7;
// put your setup code here, to run once:
for(i=0;i<3;i++)
{
    pinMode(LED[i], OUTPUT);

}
pinMode(switchpin, INPUT);
}
void loop() {
    // put your main code here, to run repeatedly:
    pulse();
    dmap(val);
    LEDcount(lev);

}

```

ULTRASONIC WITH BINARY COUNTER(using switch):

```

int LED[50];
int count = 0;
int temp = 0;
int i = 0;
int lev = 0;
int trig = 9;
int echo = 10;
long val;
long t1,t2;
int b1 = 4;//button 1
int b2 = 3;// button 2
int c1 = 0;//states of the button for detection of when button is on and for debouncing
int l1 = 0;
int c2 = 0;
int l2 = 0;
//didnt get the chance to completely check this code on the arduino, the seperate
functions however work properly
void setup() {
    // put your setup code here, to run once:

```

```
LED[0]=5;LED[1]= 6;LED[2] = 7;
// put your setup code here, to run once:
/* for(i=0;i<3;i++)
{
    pinMode(LED[i], OUTPUT);// i wanted to try and initialise the pins in a loop to
make the code shorter, didnt get proper result
```

```
    */
    pinMode(LED[0], OUTPUT);
    pinMode(LED[1], OUTPUT);
    pinMode(LED[2], OUTPUT);
    Serial.begin(9600);
    pinMode(trig, OUTPUT);
    pinMode(echo, INPUT);
    pinMode(b1, INPUT);
    pinMode(b2, INPUT);

}
void debounce(int current, int last, int switchpin)
{
    current = digitalRead(switchpin);
    if( current != last)
    {
        delay(5);current = digitalRead(switchpin);

    }
}
void LEDcount()
{
    c1 = digitalRead(b1);
    debounce(c1,l1,b1);
    if(c1 == 1 && l1 == 0)
    { if(lev>5){ lev = 0;}
    lev++;
    for(int s=0;s<3;s++)
    {
        digitalWrite( LED[s], 0);

    }
}
```



```

int a[50];
temp = lev;
Serial.print("temp: ");Serial.println(temp);
int j = 0;
while(temp != 0)
{
a[j]=temp%2;

temp = temp/2;
j++;
}
for(int r=0;r<j;r++)
{
digitalWrite( LED[r], a[r]);

Serial.print("LED[");Serial.print(r);Serial.print("]: ");//for checking states on the
serial monitor
Serial.println(digitalRead(LED[r]));
}
Serial.print("j: ");Serial.println(j);

if(temp>5){ temp = 0;}
}
l1 = c1;
delay(500);
}
void pulse()
{
digitalWrite(trig, LOW);
digitalWrite(trig, HIGH);
//delayMicroseconds(10); //ultrasonic code works with and without the delay
digitalWrite(trig, LOW);
while(digitalRead(echo)==LOW)
{}
t1 = micros();
while(digitalRead(echo) == HIGH)
{ }
t2 = micros();
val = (t2-t1)*0.034/2;

```

```
        if(val>100){Serial.println(" out of range " );}
        else
        {
            Serial.print(val);
            Serial.println("cm");
            delay(500);
        }
    }
```

```
void loop() {
    // put your main code here, to run repeatedly:
    c2 = digitalRead(b2);
    debounce(c2,l2,b2);
    if(c2==1 && l2==0)
    {
        pulse();
    }
    l2 = c2;
    LEDcount();
}
```